

Conventions and metrics

How a learning problem is written down, and how to score what a model predicts.

THE QUESTION

How do we write down a learning problem, and how do we tell whether a model's predictions are good?

THE INTUITION

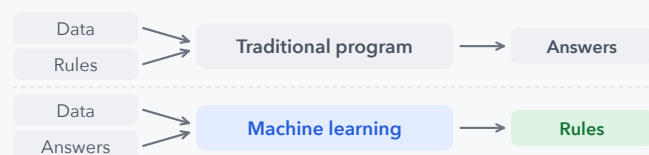
Put one sample per row and one feature per column, with the labels in a separate vector. Learn f on labelled rows, predict unseen rows, then compare the predictions with the truth. Which comparison you use depends on which mistakes cost more.

KEY TAKEAWAY

Accuracy is one number among several. On imbalanced data, read the confusion matrix, precision and recall before trusting it.

raw table → encode as numbers → X and y → learn f on train → predict test → compare with truth → pick a metric

1 Learning turns data and answers into rules



A traditional program applies rules a person wrote. Machine learning takes examples with answers and produces the rules. Mitchell: a program learns from experience E on task T , measured by P , if P improves with E . Hand rules for a digit "4" (strokes, slants, widths) never cover every way people write it.

2 A table becomes X and y

Colour	Size	Texture	Cond.		C0	C1	S0	S1	T0	T1	y
Orange	Small	Smooth	Good	x_1	0	0	1	0	1	0	1
Red	Small	Rough	Good	x_2	0	1	1	0	0	1	1
Orange	Medium	Smooth	Bad	x_3	0	0	0	1	1	0	0
Yellow	Large	Smooth	Bad	x_4	1	0	0	0	1	0	0

categories → one-hot bits
 Orange 00 · Red 01 · Yellow 10
 Small 10 · Medium 01 · Large 00
 Smooth 10 · Rough 01 · Good 1

$d = 6$ features
 $X \in \mathbb{R}^{4 \times 6}$ $y \in \mathbb{R}^4$
 $x_1 = [0, 0, 1, 0, 1, 0]^T$

Features are the inputs; the output (response) is what we predict. Categories get one-hot codes, since computers need numbers. $X \in \mathbb{R}^{n \times d}$ has row i equal to x_i^T , $y \in \mathbb{R}^n$, and $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$; here $n = 4$, $d = 6$. Bold capitals are matrices, bold lowercase vectors, plain letters scalars; $[a, b, c]^T$ is a column.

3 Train on labelled rows, predict the rest

Colour	Size	Texture	Condition	
Orange	Small	Smooth	Good	training set learn f
Red	Small	Rough	Good	
Orange	Medium	Smooth	Bad	
Yellow	Large	Smooth	Bad	
Red	Large	Rough	?	test set predict $f(x)$
Orange	Large	Rough	?	

$\mathcal{D} = \{(x_i, y_i)\}$, $i = 1..n \rightarrow y = f(x)$

The training set has features and labels; the test set has features only. Learn f on the training rows, then predict the test rows. A sample ID is not a feature (it is an arbitrary name), though it helps with auditing, such as tracing a delayed truck's batch.

4 A test set is a sample of the future



A farm has 10,000 tomatoes; you inspect 100. The 100 are a sample; the 10,000 are the population. Training and test sets are both samples of all the data the model will ever see. Generalisation means doing well on unseen data from the same distribution. A good test score only counts if the test set is representative. Dots are illustrative.

5 Classes or numbers: two kinds of task

Classification	Regression
$y_i \in \{1, \dots, k\}$	$y_i \in \mathbb{R}$
Good Bad	people temp kWh
tomato quality, loan yes/no, spam or not, tumour type	4000 30 30
	4200 30 32
	3000 20 ?
	campus energy, rainfall, house price

In classification the output is discrete, $y_i \in \{1, 2, \dots, k\}$. In regression it is continuous, $y_i \in \mathbb{R}$. Predicting campus energy (kWh) from the number of people and the temperature is regression; so is a house price. Spam or not, and malignant or benign, are classification.

6 Count the outcomes in a confusion matrix

#	actual	predicted		
1	Good	Good	● right	Good Bad 2 2 TP FP
2	Good	Good	● right	
3	Bad	Good	● wrong	
4	Bad	Good	● wrong	
5	Bad	Bad	● right	
				0 1 FN TN

accuracy = $3/5 = 0.6$

Five test tomatoes, Good = positive (1), Bad = 0. Three predictions match, so accuracy = $3/5 = 0.6$. Written with an indicator: Accuracy = $\frac{1}{n} \sum_{i=1}^n \mathbb{1}[y_i = \hat{y}_i]$. The matrix splits the two wrong ones: both are false positives (said Good, was Bad).

Which mistakes does a number hide?

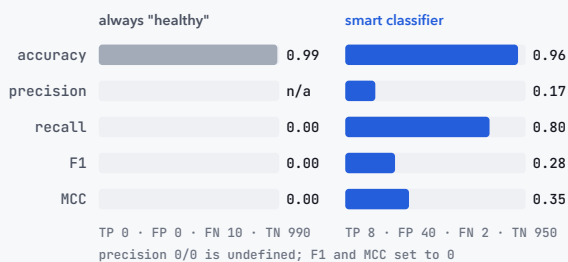
Precision and recall look at the positive class from two sides. Regression metrics average the errors in different ways.

7 Precision reads a row, recall a column



Precision = $TP / (TP + FP)$: of everything I called positive, how much was positive? **Recall** = $TP / (TP + FN)$: of all real positives, how many did I catch? For the five tomatoes: precision = $2/4 = 0.50$, recall = $2/2 = 1.00$.

8 Accuracy misleads when one class is rare



1,000 patients, 10 with the disease. A classifier that always says "healthy" scores 99% accuracy and catches nobody (recall 0). The smart classifier has lower accuracy, 95.8%, but finds 8 of the 10 cases. Fraud, spam and rare-event data behave the same way.

9 Precision or recall depends on the cost

Raising one usually lowers the other.

High precision

predicts positive only when sure; few false alarms (FP), may miss cases (FN). **Spam filter:** a lost real email costs more.

High recall

catches most positives; few misses, more false alarms. **Cancer test:** a missed case costs more.

10 F1 and MCC put both sides in one number

$$F1 = \frac{2 \cdot P \cdot R}{P + R}$$

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

F1 is the harmonic mean of precision and recall, so it stays low if either is low. MCC uses all four cells and runs from -1 to 1. For the smart classifier, $F1 = 0.28$ and $MCC = 7520 / \sqrt{48 \cdot 10 \cdot 990 \cdot 952} \approx 0.35$; the always-healthy one gets 0 on both. A multi-class problem has a $k \times k$ confusion matrix: the diagonal holds the correct predictions (see the MNIST notebook).

11 Regression metrics average the errors



With errors $e_i = \hat{y}_i - y_i$: $MSE = \frac{1}{n} \sum e_i^2$, $RMSE = \sqrt{MSE}$, $MAE = \frac{1}{n} \sum |e_i|$, $ME = \frac{1}{n} \sum e_i$. ME lets positive and negative errors cancel, so a model can look perfect while missing every point. MAE is less sensitive to outliers than MSE because it does not square: errors 1, 1, 1, 1, 10 give MAE 2.8 but RMSE 4.6 (illustrative).

12 The same numbers in scikit-learn

```
from sklearn.metrics import (
    confusion_matrix, precision_score,
    recall_score, f1_score,
    matthews_corrcoef)

# y, y_hat: 0/1 arrays, positive class = 1
precision_score(y, y_hat) # 8/48 = 0.17
recall_score(y, y_hat)   # 8/10 = 0.80
f1_score(y, y_hat)       # 0.28
matthews_corrcoef(y, y_hat) # 0.35
```

`confusion_matrix(y, y_hat)` puts actual classes in rows and predicted in columns, the transpose of the lecture's layout. Check which way a matrix is drawn before reading FP and FN off it.

13 Pause and think

Is the sample number a feature?

Usually not; it is an arbitrary identifier.

Why prefer MAE to MSE? It is less sensitive to outliers, since it does not square errors.

Which is regression: spam, cat/dog/bird, house price, tumour type?

House price, a continuous value.

Imbalanced data: which metrics?

Precision, recall and F1, read with the confusion matrix.

Always predict "negative" on 1,000 samples with 10 positives: accuracy? 99%, and the model is useless.

Is a good test score enough? Only if the test set represents the data the model will meet.

rows are samples, columns features · learn on train, judge on test · match the metric to the cost of a mistake

Source: lecture slides "Conventions, accuracy metrics, classification, regression" (slides); numbers are the lecture's tables, with metrics recomputed in NumPy. The slide's MCC for the smart classifier (0.26) is corrected here to 0.35.